Graph Theroy

Ashan De Silva

2026-09-05

Contents

Chapter 1

Introduction to Graph Theory

1.1 What is Graph Theory?

At its core, **graph theory** is the study of relationships. Whenever we have a collection of objects and connections between them, we can use a graph to model, analyze, and solve problems involving that structure.

Unlike the graphs you might recall from introductory algebra (like $y = x^2$ or sine waves), a graph in discrete mathematics is a structural map. It consists of individual points and the lines that connect them.

```
(v1) ------------ (v2)
 |                 |
 |                 |
(v4) ------------ (v3)
```

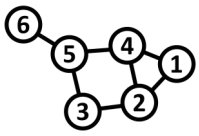


Figure 1.1: A graph with 6 vertices and edges *(Image Credit: Wikipedia)*

```
The objects are represented as points called <strong>vertices</strong> (or <strong>nodes</strong>
and the connections between them are called <strong>edges</strong>.
```

```
<img src="fig/fig 2.png" style="width:100%;">
```

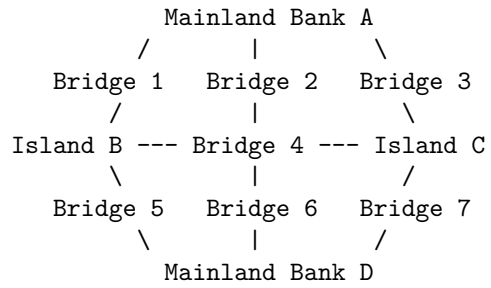
Graphs give us a unified mathematical language to represent networks. Whether we are analyzing social networks, computer communication grids, or transit maps, graph theory provides the core framework to understand how systems are connected.

1.2 Historical Foundations

To appreciate graph theory, it helps to understand the historical puzzles that sparked its development.

1.2.1 The Seven Bridges of Königsberg (1736)

Graph theory was born in the Prussian city of Königsberg (now Kaliningrad, Russia). The city was divided by the Pregel River, creating two large islands connected to each other and to the mainland riverbanks by seven bridges.



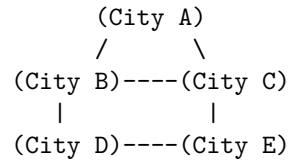
The citizens posed a classic recreational puzzle: *Is it possible to take a walk through the city in such a way that you cross every single one of the seven bridges exactly once, and end up back where you started?*

In 1736, the Swiss mathematician **Leonhard Euler** solved this problem. He realized that the specific distances, land shapes, and physical geography did not matter; only the **connections** mattered. Euler simplified the map by replacing each landmass with a point (vertex) and each bridge with a line (edge).

By abstracting the geography into a mathematical structure, Euler proved that such a walk was impossible. In doing so, he laid the foundation for topology and graph theory.

1.2.2 Hamilton's Icosian Game (1859)

Over a century later, in 1859, Irish mathematician **Sir William Rowan Hamilton** created a commercial puzzle called the *Icosian Game*.



The game used a solid wooden regular dodecahedron—a 12-faced polyhedron with 20 corners. Each of the 20 corners was labeled with the name of a famous world city. The objective was to find a route along the edges of the dodecahedron that visited **every city exactly once** and returned to the starting point.

While Euler's bridge problem focused on visiting every *edge* once, Hamilton's game focused on visiting every *vertex* once. These two historical problems led directly to two major areas of graph theory: **Eulerian paths** and **Hamiltonian cycles**.

1.3 Modern Applications

Today, graph theory is an essential foundation for computer science, data analytics, and operational engineering.

- **Social Networks**: Platforms model users as vertices and friendships or followers as edges. Graph theory algorithms help identify tight-knit communities, influential users, and recommendation loops.
- **Transportation Networks**: Airline routes, train networks, and road grids are modeled as graphs. GPS platforms rely on these models to compute optimal routes.
- **The Web & PageRank**: The World Wide Web is a massive graph where web pages are vertices and hyperlinks are directed edges. Early search engines transformed web searches by using graph structure to rank relevance based on link density.

1.4 Formal Definitions and Basic Structures

Let us establish the formal mathematical definitions used throughout graph theory.

1.4.1 Mathematical Definition of a Graph

A **graph** G is a pair $G = (V, E)$, where:

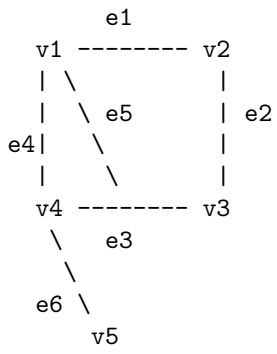
- V is a non-empty set of **vertices** (or nodes).
- E is a set of **edges**, where each edge $e \in E$ is associated with an unordered pair of vertices $\{u, v\} \subseteq V$.

If an edge e connects vertices u and v, we write $e = (u, v)$ or $e = uv$. We say that e is **incident** with u and v, and that u and v are **adjacent** (or neighbors).

1.4.1.1 Example

Consider a graph $G = (V, E)$ defined by:

- $V = \{v_1, v_2, v_3, v_4, v_5\}$
- $E = \{e_1, e_2, e_3, e_4, e_5, e_6\}$ where:
- $e_1 = (v_1, v_2)$
- $e_2 = (v_2, v_3)$
- $e_3 = (v_3, v_4)$
- $e_4 = (v_4, v_1)$
- $e_5 = (v_1, v_3)$
- $e_6 = (v_4, v_5)$



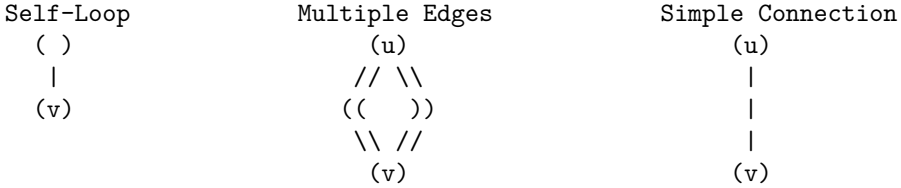
In this graph:

- v_1 and v_3 are adjacent because $e_5 = (v_1, v_3)$ connects them.
- v_2 and v_5 are **not** adjacent because there is no edge directly connecting them.

1.4.2 Loops, Multiple Edges, and Simple Graphs

Not all graphs are strictly constructed with single lines between distinct points.

- **Self-loop**: An edge that joins a vertex to itself (e.g., $e = (v_1, v_1)$).
- **Multiple Edges (Parallel Edges)**: A collection of two or more edges that connect the exact same pair of distinct end vertices.
- **Simple Graph**: A graph that contains **no self-loops and no multiple edges**.



Unless specified otherwise, standard graph theory problems assume a graph is **simple**.

1.5 Vertex Degrees and Handshaking

1.5.1 Vertex Degree

The **degree** of a vertex v, denoted by $d(v)$, is the number of edges incident to v. For simple graphs, it is equal to the number of neighbors connected to v.

(Note: If a vertex has a self-loop, that loop contributes 2 *to the vertex's degree, as it touches the vertex twice.)*

1.5.1.1 Example

Using Example 1.1 above:

- $d(v_1) = 3$ (connected by e_1, e_4, e_5)
- $d(v_2) = 2$ (connected by e_1, e_2)
- $d(v_3) = 3$ (connected by e_2, e_3, e_5)
- $d(v_4) = 3$ (connected by e_3, e_4, e_6)
- $d(v_5) = 1$ (connected by e_6)

Summing all degrees: $3 + 2 + 3 + 3 + 1 = 12$. Notice that the total number of edges $|E|$ is 6. The sum of degrees (12) is exactly twice the number of edges (2×6).

1.5.2 The Handshaking Lemma

This observation holds true for all graphs.

Theorem (Handshaking Lemma):

Let $G = (V, E)$ be any graph. Then the sum of the degrees of all vertices is equal to twice the number of edges:

$$\sum_{v \in V} d(v) = 2|E|$$

Proof Intuition:

Every edge $e = (u, v)$ has two endpoints. When we count the degree of vertex u, edge e contributes 1. When we count the degree of vertex v, edge e contributes 1 again. Therefore, every edge contributes exactly 2 to the sum total of degrees in the graph.

1.5.2.1 Corollary: The Parity of Odd-Degree Vertices

In any graph G, the number of vertices that have an **odd degree** must be **even**.

Proof:

Partition the vertex set V into two subsets: V_{even} (vertices with even degree) and V_{odd} (vertices with odd degree).

$$\sum_{v \in V} d(v) = \sum_{v \in V_{\text{even}}} d(v) + \sum_{v \in V_{\text{odd}}} d(v) = 2|E|$$

1. The total sum $2|E|$ is always an even integer.
2. The sum $\sum_{v \in V_{\text{even}}} d(v)$ is a sum of even integers, so it must be even.
3. For the equation to balance, the sum $\sum_{v \in V_{\text{odd}}} d(v)$ must also be **even**.
4. A sum of odd integers is even if and only if there is an **even number** of odd integers.

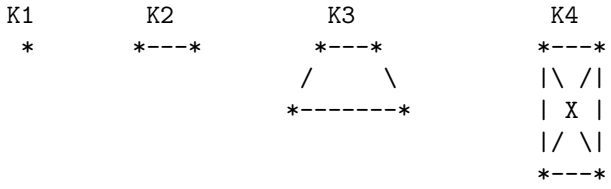
Thus, the number of vertices with odd degree is always even.

1.6 Special Classes of Graphs

Certain graph structures appear frequently throughout graph theory and computer science applications.

1.6.1 1. Complete Graphs (K_n)

A **complete graph** on n vertices, denoted by K_n, is a simple graph in which every pair of distinct vertices is connected by an edge.



- **Number of Edges in K_n**: To find the total number of edges, we count how many ways we can pick a pair of vertices from n available vertices:

$$|E| = \binom{n}{2} = \frac{n(n-1)}{2}$$

1.6.2 2. Regular Graphs

A graph is k**-regular** if every vertex has the same degree k.

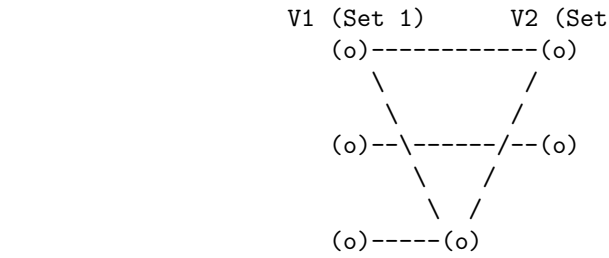
- In a k-regular graph with n vertices, $\sum d(v) = n \cdot k$. By the Handshaking Lemma:

$$n \cdot k = 2|E| \implies |E| = \frac{n \cdot k}{2}$$

- **Note**: A complete graph K_n is an $(n-1)$-regular graph.

1.6.3 3. Bipartite Graphs

A graph $G = (V, E)$ is **bipartite** if its vertex set V can be partitioned into two disjoint subsets, V_1 and V_2 ($V = V_1 \cup V_2$ and $V_1 \cap V_2 = \emptyset$), such that every edge in E connects a vertex in V_1 to a vertex in V_2. No edges exist between two vertices within V_1 or within V_2.



1.6.3.1 Complete Bipartite Graphs (K_{n_1,n_2})

A **complete bipartite graph** K_{n_1,n_2} is a bipartite graph where every vertex in V_1 (with $|V_1| = n_1$) is connected to every vertex in V_2 (with $|V_2| = n_2$).

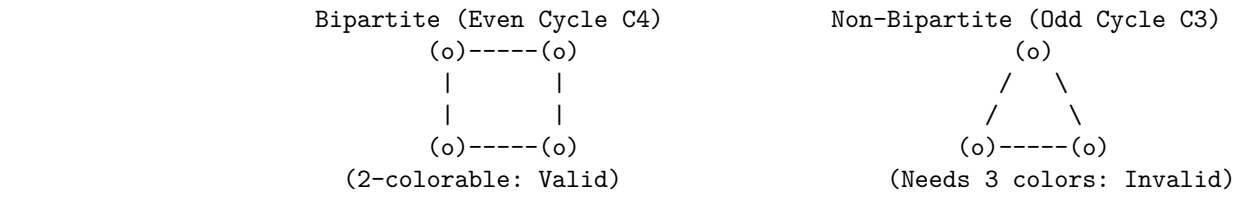
- Total Vertices: $|V| = n_1 + n_2$
- Total Edges: $|E| = n_1 \cdot n_2$

1.6.3.2 Characterization Theorem of Bipartite Graphs

How can we determine if a given graph is bipartite?

Theorem:

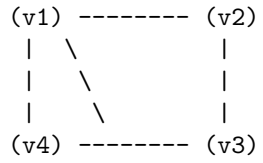
A graph G is bipartite **if and only if** it contains **no cycles of odd length**.



- If a graph contains a triangle (C_3), a pentagon (C_5), or any other cycle with an odd number of edges, it cannot be partitioned into two valid bipartite sets.

1.7 Traversal Terminology: Walks, Trails, Paths, Circuits, and Cycles

To navigate through a graph, we need clear definitions for moving along vertices and edges.



1.7.1 Definitions

1. **Walk**: A sequence of alternating vertices and edges, starting at vertex v_0 and ending at v_k: $v_0, e_1, v_1, e_2, \dots, e_k, v_k$. Vertices and edges **may be repeated**.
2. **Trail**: A walk in which **no edge is repeated**. Vertices may still be repeated.
3. **Path**: A trail in which **no vertex is repeated**. (Consequently, no edge is repeated either).
4. **Closed Walk / Trail**: A walk or trail that starts and ends at the exact same vertex ($v_0 = v_k$).
5. **Circuit**: A **closed trail** (starts and ends at the same vertex, with no repeated edges).
6. **Cycle**: A **closed path** (starts and ends at the same vertex, with no repeated vertices other than the start and end).

1.7.2 Summary Table

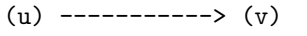
Structure	Repeated Vertices Allowed?	Repeated Edges Allowed?	Start and End at Same Vertex?
Walk	Yes	Yes	Optional
Trail	Yes	**No**	Optional

Structure	Repeated Vertices Allowed?	Repeated Edges Allowed?	Start and End at Same Vertex?
Path	**No**	**No**	No
Circuit	Yes	**No**	**Yes**
Cycle	**No** (except start/end)	**No**	**Yes**

1.8 Directed Graphs (Digraphs)

In many real-world networks, connections are one-way streets. For example, a web page may link to another, but that page might not link back.

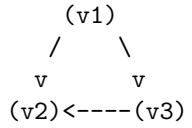
A **directed graph** (or **digraph**) consists of a vertex set V and an edge set E containing **ordered pairs** of vertices. An edge $e = (u, v)$ goes **from** u **to** v.



1.8.1 In-Degree and Out-Degree

Because edges have direction in a digraph, we split vertex degree into two separate counts:

- **In-degree ($d_{\text{in}}(v)$)**: The number of directed edges coming **into** vertex v.
- **Out-degree ($d_{\text{out}}(v)$)**: The number of directed edges going **out of** vertex v.



In the example above:

- $d_{\text{in}}(v_1) = 0$, $d_{\text{out}}(v_1) = 2$
- $d_{\text{in}}(v_2) = 2$, $d_{\text{out}}(v_2) = 0$
- $d_{\text{in}}(v_3) = 1$, $d_{\text{out}}(v_3) = 1$

1.8.1.1 Handshaking Lemma for Directed Graphs

In any directed graph, the sum of all in-degrees equals the sum of all out-degrees, which both equal the total number of directed edges:

$$\sum_{v \in V} d_{\text{in}}(v) = \sum_{v \in V} d_{\text{out}}(v) = |E|$$

1.9 Chapter 1 Exercises

1. **Degree Calculations**: A simple graph has 8 vertices and 14 edges. If 4 vertices have degree 4, and 3 vertices have degree 3, what is the degree of the remaining vertex?
2. **Bipartite Verification**: Prove or disprove whether a single cycle graph C_6 (a cycle with 6 vertices) is bipartite. What about C_7?
3. **Complete Graph Proof**: Use induction to show that the number of edges in a complete graph K_n is given by $\frac{n(n-1)}{2}$.
4. **Handshaking Lemma**: Is it possible to construct a simple graph with 5 vertices such that their degrees are $3, 3, 3, 2, 1$? Explain using the Handshaking Lemma.

Chapter 2

Introduction to Graph Theory

2.0.1 Core Concepts & The Seven Bridges Problem

- **Königsberg Seven Bridges Problem**: A foundational historical problem solved by Leonhard Euler in 1736, laying the groundwork for modern graph theory. The goal was to find a walk through the city crossing each of its seven bridges exactly once.

Image Source: Wikipedia

Figure 2.1: Image Source: Wikipedia

1. Historical Foundations and Applications

- **Seven Bridges of Königsberg**: Graph theory originated with Leonhard Euler's attempt to find a continuous path across the seven bridges of Königsberg that crossed each bridge exactly once (an Eulerian path).
- **Hamiltonian Cycles**: In 1859, Sir William Rowan Hamilton invented a game based on a wooden regular dodecahedron with 20 vertices representing major cities. The goal was to construct a closed cycle along the edges visiting every vertex precisely once (a Hamiltonian cycle).
- **Modern Applications**:
- **Community Detection**: Uncovering hierarchical substructures within complex networks.

- **Transportation and Infrastructure**: Modeling complex systems such as large-scale transit routes and the Internet.
- **Algorithmics**: Ranking hyperlinks and computing shortest paths in GPS navigation.

---

2. Basic Definitions and Graph Structural Types

- **Graph (G)**: Defined as an ordered pair $G = (V, E)$, where V is a non-empty set of vertices (or nodes) and E is a set of edges connecting vertex pairs. An edge $e_k = (v_i, v_j)$ is incident with vertices v_i and v_j.
- **Self-loops & Multiple Edges**:
- **Self-loop**: An edge connecting a vertex to itself.
- **Multiple edges**: Two or more edges joining the same distinct endpoints.
- **Simple Graph**: A graph containing no self-loops or multiple edges.
- **Directed Graph (Digraph)**: A graph where each edge possesses a direction. Every vertex v is characterized by an in-degree $d_{\text{in}}(v)$ and an out-degree $d_{\text{out}}(v)$.

---

3. Degree Properties and Structural Classes

- **Handshaking Lemma**: The sum of degrees across all vertices equals twice the total number of edges:

$$\sum_{v \in V} d(v) = 2|E|$$

- **Parity Corollary**: Any graph G contains an even number of odd-degree vertices.
- **Complete Graph (K_n)**: A simple graph with n vertices where every pair of distinct vertices is connected by an edge, containing $\frac{n(n-1)}{2}$ edges.
- **k-Regular Graph**: A simple graph where every vertex has degree k. K_n is $(n-1)$-regular.
- **Bipartite Graph**: A graph whose vertex set can be partitioned into $V = V_1 \cup V_2$ such that every edge connects a vertex in V_1 to a vertex in V_2.

- **Complete Bipartite Graph (K_{n_1,n_2})**: Contains all $n_1 \cdot n_2$ possible edges between V_1 and V_2.
- **Characterization**: A graph is bipartite **if and only if** it contains no odd-length cycles.

---

4. Walk Invariants and Traversal Definitions

Structure	Vertices	Edges	Endpoint Condition
Walk	May repeat	May repeat	Open
Trail	May repeat	Distinct	Open
Path	Distinct	Distinct	Open
Circuit	May repeat	Distinct	Closed ($v_0 = v_k$)
Cycle	Distinct (except endpoints)	Distinct	Closed ($v_0 = v_k$)

- **Basic Definitions**:
- **Graph** $G = (V, E)$: A set of vertices (nodes) V and a set of edges E connecting pairs of vertices.
- **Degree** $d(v)$: The number of edges incident to vertex v.
- **Walk**: An alternating sequence of vertices and edges.
- **Trail**: A walk with no repeated edges.
- **Path**: A walk with no repeated vertices.
- **Cycle**: A closed path where the first and last vertices are identical.

2.1 Lecture 2: Graph Types, Operations & Representations

2.1.1 Graph Isomorphism & Counting

- **Isomorphism**: Two graphs are isomorphic if there exists a 1-to-1 correspondence between their vertex sets that preserves adjacencies and non-adjacencies. Finding if two graphs are isomorphic is believed to be an NP-hard problem.
- **Counting Simple Graphs**:
- Maximum possible edges in a simple graph with n vertices: $\frac{n(n-1)}{2}$.

- Total possible labeled graphs with n vertices: $2^{n(n-1)/2}$.
- The number of non-isomorphic graphs T_n satisfies: $\frac{2^{n(n-1)/2}}{n!} \leq T_n \leq 2^{n(n-1)/2}$.

2.1.2 Graph Transformations

- **Complement Graph** $\overline{G}$: Given $G = (V, E)$, $\overline{G} = (V, E')$ shares the same vertex set, but its edges are precisely those missing from G.
- **Line Graph** $L(G)$: Represents edge adjacencies in G. Each vertex in $L(G)$ corresponds to an edge in G, and two vertices in $L(G)$ are adjacent if their respective edges in G share a common vertex.

```
Original Edge (u, v) ---- Original Edge (v, w)
          \                        /
           --- Line Graph Node --
```

2.1.3 Eulerian & Hamiltonian Graphs

- **Eulerian Trail**: A trail containing every edge of G.
- **Eulerian Tour**: A closed Eulerian trail.
- **Euler's Theorem**: A connected graph has an Euler tour **if and only if** every vertex has an **even degree**.
- **Hamiltonian Cycle**: A cycle that visits every **vertex** of a graph exactly once.

2.1.4 Graph Representations

- **Adjacency Matrix (A)**: An $n \times n$ matrix where $A(i, j) = 1$ if $(i, j) \in E$, else 0.
- **Incidence Matrix (T)**: An $n \times m$ matrix (vertices $\times$ edges):
- Undirected: $T(i, k) = 1$ if vertex i is incident with edge k.
- Directed: $T(i, k) = -1$ if arc k leaves vertex i; $T(j, k) = 1$ if arc k enters vertex j.
- **Adjacency List**: Sparse matrix structure mapping each node to its list of outward neighbors.

2.2 Lecture 3: Connectivity & Graph Exploration

2.2.1 Connectivity

- **Connected Component**: Equivalence classes formed by the walk relation $u \sim v$ on undirected graphs.
- **Strong Connectivity (Digraphs)**: u and v are strongly connected if there exists a directed walk from u to v **and** from v to u.
- **Strongly Connected Component (SCC)**: $C(u) = R_+(u) \cap R_-(u)$, where $R_+(u)$ is reachable from u, and $R_-(u)$ can reach u. Reaching set $R_-(u)$ is found using the inverse graph G^T (represented by transpose adjacency matrix A^T).

2.2.2 Exploration Algorithms

Algorithm	Queue/Data Structure	Strategy	Complexity
Generic Search	Set L	Visits arbitrary unvisited neighbor	

| $O(V)$ steps (linear)

| | **DFS** | LIFO Stack

| Explores deeply along branches before backtracking

| $O(V + E)$ | | **BFS** | FIFO Queue

| Explores all immediate neighbors level-by-level

| $O(V + E)$ |

2.3 Lecture 4: Planar Graphs

2.3.1 Definitions & Euler's Formula

- **Planar Graph**: A graph that can be embedded (drawn) in a 2D plane such that no edges intersect except at their endpoints.

- **Fáry's Theorem**: Every planar graph can be drawn using straight line segments.
- **Euler's Formula**: For any plane-embedded graph with v vertices, e edges, r regions, and c components:

$$v - e + r = c + 1$$

For connected graphs ($c = 1$), this simplifies to $v - e + r = 2$.

2.3.2 Invariants & Non-Planarity Criteria

- **Edge Bounds**:
- Simple planar graph ($v \geq 3$): $e \leq 3v - 6$.
- Planar graph with no cycle of length < 4 (girth ≥ 4): $e \leq 2v - 4$.
- **Degree Invariants**:
- Average vertex degree in a planar graph: $< 6 - \frac{12}{n}$.
- Every planar graph contains at least one vertex with degree $d(v) \leq 5$.
- **Kuratowski's Theorem**: A graph is non-planar **if and only if** it contains a subdivision of K_5 (complete graph on 5 vertices) or $K_{3,3}$ (complete bipartite graph on 3+3 vertices).

2.4 Lecture 5 & 6: Trees, Spanning Trees & Minimum Spanning Trees (MST)

2.4.1 Tree Properties

- **Tree**: A connected, acyclic graph.
- **Forest**: An acyclic collection of trees. A forest on n vertices with c components has $n - c$ edges.
- **Equivalent Tree Definitions** ($|V| = n$):

1. Connected and acyclic.

2. Connected with $n-1$ edges.

3. Acyclic with $n-1$ edges.

4. Unique path between any pair of vertices.

- **Cayley's Formula**: Number of distinct labeled trees on n vertices is n^{n-2}.

2.4.2 Rooted Trees

- **Terminology**: Root, Parent, Child, Siblings, Leaf (no children), Depth (distance to root), Height (maximum depth).

- m**-ary Tree**: Every node has at most m children.

- **Center of a Tree**: Single vertex or pair of adjacent vertices that minimizes maximum distance to any other node (eccentricity).

2.4.3 Minimum Spanning Tree Algorithms

```
     Kruskal's Algorithm
[Sort all edges by weight]
            |
[Add edge if no cycle formed]
            |
[Repeat until V-1 edges]
```

```
      Prim's Algorithm
[Start from single node u]
             |
 [Add minimum edge connecting]
 [visited to unvisited set]
           |
 [Repeat until V-1 edges]
```

- **Kruskal's Algorithm**: Edge-centric greedy approach. Sorts all edges by weight and iteratively adds the minimum-weight edge that does not form a cycle.

- **Prim's Algorithm**: Vertex-centric greedy approach. Starts with a single vertex and iteratively expands the tree by adding the minimum-weight edge connecting the current tree to an unvisited vertex.

2.5 Lecture 7 & 8: Graph Coloring & Applications

2.5.1 Definitions & Results

- **k-Coloring**: Assignment $C : V \to \{1, \dots, k\}$ such that $C(u) \neq C(v)$ for all $uv \in E$.
- **Chromatic Number** $\chi(G)$: Minimal k required to color G.
- **Key Chromatic Values**:
- Bipartite Graphs: $\chi(G) = 2$
- Cycles (C_n): $\chi(C_n) = 2$ (even n), $\chi(C_n) = 3$ (odd n)
- Cliques (K_n): $\chi(K_n) = n$
- Four-Color Theorem: Every planar graph has $\chi(G) \leq 4$.

2.5.2 Applications of Graph Coloring

1. **Conflict Allocation (e.g., Tropical Fish Tank Allocation)**: Vertices = entities (fish); Edges = conflicts; Colors = tanks required.
2. **Scheduling Problems (e.g., Exam Scheduling)**: Vertices = courses; Edges = student overlap conflicts; Colors = minimum time slots needed.

2.6 Lecture 9: Shortest Path Algorithms

2.6.1 Algorithms Taxonomy

Problem Type	Algorithm	Strategy	Constraints
Unweighted	BFS Exploration		

| Level-by-level queue traversal

| All edge weights equal 1 | | **Weighted (Non-negative)** | Dijkstra's Algorithm

| Greedy search via priority queue

| Requires non-negative edge weights $w(e) \geq 0$ | | **Negative Weights** | Modified Dijkstra / Bellman-Ford | Iterative relaxation | Detects negative weight cycles |

- **Dijkstra's Relaxation Step**:

$$\text{If } \lambda(v) > \lambda(u) + w(u,v) \implies \lambda(v) = \lambda(u) + w(u,v)$$

2.7 Lecture 10: Networks and Flows

2.7.1 Network Definitions

- **Network**: A directed graph with source s, sink t, and arc capacities $c_{i,j} \geq 0$.
- **Flow Constraints**:

1. Capacity Constraint: $0 \leq x_{i,j} \leq c_{i,j}$
2. Conservation (Kirchhoff's Law): $\sum_k x_{k,i} = \sum_k x_{i,k}$ for all vertices except source s and sink t.

2.7.2 Ford-Fulkerson Algorithm & Max-Flow Min-Cut

1. Initialize flow $x_{i,j} = 0$ everywhere.
2. Search for an **Augmenting Path** from source to sink with bottleneck capacity $\Delta > 0$.
3. Increase path flow by Δ and repeat until no augmenting path exists.

- **Max-Flow Min-Cut Theorem**: The maximum flow value in a network equals the minimum capacity of a cut separating s and t.

2.8 Lecture 11 & 12: Matchings & Covers

2.8.1 Matchings Definitions

- **Matching (M)**: A subset of edges in G such that no two edges share a common vertex.
- **Maximal Matching**: A matching that cannot be extended by adding any remaining edge.
- **Maximum Matching**: A matching with the highest possible cardinality ($\nu(G)$).
- **Perfect Matching**: A matching that covers every vertex in V (deficiency $= 0$).
- **Augmenting Path**: An M-alternating path whose endpoints are M-exposed. A matching M is maximum **if and only if** it has no M-augmenting paths.

2.8.2 Graph Covers

- **Line Cover**: Edge subset covering every vertex (requires $d(v) \geq 1$).
- **Vertex Cover**: Vertex subset covering every edge in E.

2.8.3 Fundamental Matching Theorems

- **Hall's Marriage Theorem**: A bipartite graph with sets X, Y has a matching covering X **if and only if** for every subset $S \subseteq X$, its neighborhood satisfies $|N(S)| \geq |S|$.
- **Tutte-Berge Formula**:

$$\nu(G) = \min_{U \subseteq V} \frac{1}{2} \left(|V| - \alpha(G \setminus U) + |U| \right)$$

where $\alpha(G \setminus U)$ denotes the number of odd connected components in $G \setminus U$.

Chapter 3

Cross-references

Cross-references make it easier for your readers to find and link to elements in your book.

3.1 Chapters and sub-chapters

There are two steps to cross-reference any heading:

1. Label the heading: `# Hello world {#nice-label}`.
    - Leave the label off if you like the automated heading generated based on your heading title: for example, `# Hello world` = `# Hello world {#hello-world}`.
    - To label an un-numbered heading, use: `# Hello world {-#nice-label}` or `{# Hello world .unnumbered}`.
2. Next, reference the labeled heading anywhere in the text using `\@ref(nice-label)`; for example, please see Chapter 3.
    - If you prefer text as the link instead of a numbered reference use: any text you want can go here.

3.2 Captioned figures and tables

Figures and tables *with captions* can also be cross-referenced from elsewhere in your book using `\@ref(fig:chunk-label)` and `\@ref(tab:chunk-label)`, respectively.

See Figure 3.1.

```
par(mar = c(4, 4, .1, .1))
plot(pressure, type = 'b', pch = 19)
```

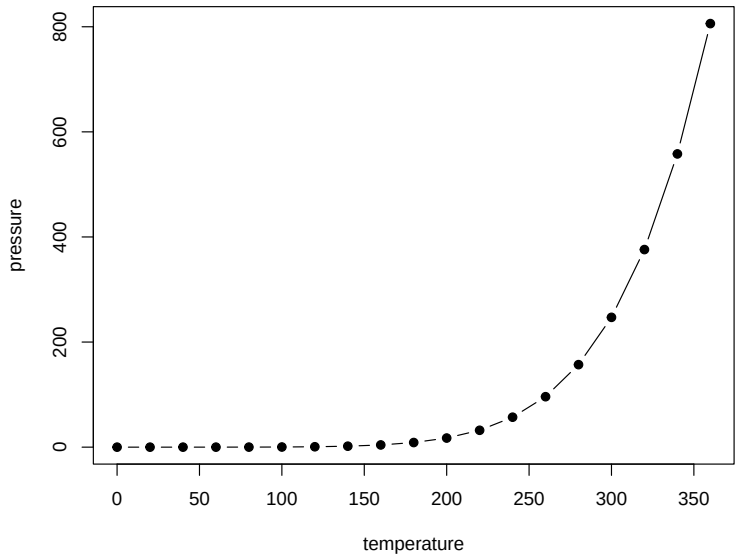


Figure 3.1: Here is a nice figure!

Don't miss Table 3.1.

```
knitr::kable(
  head(pressure, 10), caption = 'Here is a nice table!',
  booktabs = TRUE
)
```

Table 3.1: Here is a nice table!

temperature	pressure
0	0.0002
20	0.0012
40	0.0060
60	0.0300
80	0.0900
100	0.2700
120	0.7500
140	1.8500
160	4.2000
180	8.8000

Chapter 4

Parts

You can add parts to organize one or more book chapters together. Parts can be inserted at the top of an .Rmd file, before the first-level chapter heading in that same file.

Add a numbered part: `# (PART) Act one {-}` (followed by `# A chapter`)

Add an unnumbered part: `# (PART\*) Act one {-}` (followed by `# A chapter`)

Add an appendix as a special kind of un-numbered part: `# (APPENDIX) Other stuff {-}` (followed by `# A chapter`). Chapters in an appendix are prepended with letters instead of numbers.

Chapter 5

Footnotes and citations

5.1 Footnotes

Footnotes are put inside the square brackets after a caret `^[]`. Like this one [1].

5.2 Citations

Reference items in your bibliography file(s) using `@key`.

For example, we are using the **bookdown** package [Xie, 2026] (check out the last code chunk in index.Rmd to see how this citation key was added) in this sample book, which was built on top of R Markdown and **knitr** [Xie, 2015] (this citation was added manually in an external file book.bib). Note that the `.bib` files need to be listed in the index.Rmd with the YAML `bibliography` key.

The RStudio Visual Markdown Editor can also make it easier to insert citations: https://rstudio.github.io/visual-markdown-editing/#/citations

[1] This is a footnote.

Chapter 6

Blocks

6.1 Equations

Here is an equation.

$$f(k) = \binom{n}{k} p^k (1-p)^{n-k} \tag{6.1}$$

You may refer to using `\@ref(eq:binom)`, like see Equation (6.1).

6.2 Theorems and proofs

Labeled theorems can be referenced in text using `\@ref(thm:tri)`, for example, check out this smart theorem 6.1.

Theorem 6.1. *For a right triangle, if c denotes the* length *of the hypotenuse and a and b denote the lengths of the* ***other*** *two sides, we have*

$$a^2 + b^2 = c^2$$

Read more here https://pkg.yihui.org/bookdown/markdown-extensions-by-bookdown.html.

6.3 Callout blocks

The R Markdown Cookbook provides more help on how to use custom blocks to design your own callouts: https://yihui.org/rmarkdown-cookbook/custom-blocks.html

Chapter 7

Sharing your book

7.1 Publishing

HTML books can be published online, see: https://pkg.yihui.org/bookdown/publishing.html

7.2 404 pages

By default, users will be directed to a 404 page if they try to access a webpage that cannot be found. If you'd like to customize your 404 page instead of using the default, you may add either a `_404.Rmd` or `_404.md` file to your project root and use code and/or Markdown syntax.

7.3 Metadata for sharing

Bookdown HTML books will provide HTML metadata for social sharing on platforms like Twitter, Facebook, and LinkedIn, using information you provide in the `index.Rmd` YAML. To setup, set the `url` for your book and the path to your `cover-image` file. Your book's `title` and `description` are also used.

This `gitbook` uses the same social sharing data across all chapters in your book- all links shared will look the same.

Specify your book's source repository on GitHub using the `edit` key under the configuration options in the `_output.yml` file, which allows users to suggest an edit by linking to a chapter's source file.

Read more about the features of this output format here:

https://pkgs.rstudio.com/bookdown/reference/gitbook.html

Or use:

```
?bookdown::gitbook
```

Bibliography

Yihui Xie. *Dynamic Documents with R and knitr*. Chapman and Hall/CRC, Boca Raton, Florida, 2nd edition, 2015. URL http://yihui.org/knitr/. ISBN 978-1498716963.

Yihui Xie. *bookdown: Authoring Books and Technical Documents with R Markdown*, 2026. URL https://github.com/rstudio/bookdown. R package version 0.47.